LQR 转向平衡控制模块

> 基于 Cornell 点质量模型的嵌入式 LQR 平衡控制器
>
> 可移植 · 无依赖 · 易集成

1. 目录结构

```
lqr_module/
├── driver/                    # 驱动层（核心算法，无依赖）
│   ├── lqr_driver.h          # 接口定义
│   └── lqr_driver.c          # 实现
├── app/                       # 应用层（参考实现）
│   ├── lqr_balance.h         # 接口定义
│   └── lqr_balance.c         # 实现
├── tools/                     # 工具
│   └── lqr_gain_generator.py # 增益表生成器
└── README.md                  # 本文件
```

Fence 1

2. 快速开始

2.1 1. 复制文件

将 `driver/` 和 `app/` 文件夹复制到你的项目中。

2.2 2. 实现平台接口

在你的代码中实现以下 4 个函数（在 `lqr_balance.h` 中声明为 `extern`）：

```c
// 获取电机转速（RPM）
int motor_get_speed(void) {
    return your_motor_rpm;
}

// 获取侧倾角（度）
float imu_get_roll(void) {
    return your_imu_roll_deg;
}

// 获取侧倾角速度（度/秒）
float imu_get_roll_rate(void) {
    return your_imu_gyro_y;
}

```

```
// 设置舵机PWM
void servo_set(uint32_t pwm) {
    your_servo_write(pwm);
}
```

Fence 2

2.3 3. 初始化与调用

```
#include "lqr_balance.h"
// 初始化LQR平衡控制（内部会设置物理参数和限幅）
    lqr_balance_init();

// 在定时中断中调用（推荐 200Hz / 5ms）
    lqr_balance_control();
```

Fence 3

`lqr_balance_init()` 和 `lqr_balance_control()` 已经封装好了所有逻辑。

2.4 4. 修改物理参数

在 `lqr_balance.c` 的 `lqr_balance_init()` 中修改：

```
/* 设置物理参数（需根据实际车辆测量后调整） */
lqr_set_physical_params(&lqr_ctrl,
    0.12f,   // h: 质心高度 (m)
    0.21f,   // L: 轴距 (m)
    0.02f);  // b: 拖曳距 (m)
```

Fence 4

3. 参数配置

3.1 物理参数（必须实测）

参数	符号	典型值	测量方法
质心高度	h	0.10~0.15m	悬挂法
轴距	L	0.15~0.25m	直接测量
拖曳距	b	0.01~0.03m	几何测量

Table 1

3.2 舵机参数

在 `lqr_balance.h` 中修改：

```
#define LQR_SERVO_MID           1500    // 舵机中位PWM (us)
#define LQR_SERVO_LEFT          1100    // 左极限
#define LQR_SERVO_RIGHT         1900    // 右极限
#define LQR_DELTA_MECHANICAL_MAX    DEG_RAD(23.0f)  // 最大机械转向角
```

Fence 5

4. 增益表生成工具（GUI）

4.1 安装依赖

```
pip install numpy scipy
```

Fence 6

> 注意：tkinter 是 Python 标准库，无需单独安装。Windows 自带，Linux 可能需要 `sudo apt install python3-tk`

4.2 运行 GUI 工具

```
cd tools
python lqr_gain_generator.py
```

Fence 7

启动后会弹出图形界面，可以：

1. 调整物理参数（h, w, b）
2. 设置 LQR 权重（Q, R）
3. 设置速度范围
4. 点击「计算」生成增益表
5. 点击「复制代码」将 C 代码复制到剪贴板
6. 粘贴到 `lqr_driver.c` 的 `default_gain_entries[]` 数组

4.3 GUI 界面功能

区域	说明
物理参数	质心高度、轴距、拖曳距
LQR权重	Q[0]~Q[2] 和 R 的值
速度范围	最小/最大速度、速度点数

Table 2

5. 调参指南

现象	调整方向
倒地前舵机反应慢	减小 R 或增大 Q_phi
舵机疯狂抖动	增大 R 或加强 IMU 滤波

Table 3

现象	调整方向
能平衡但蛇行	增大 Q_delta
振荡不收敛	增大 Q_phi_dot

5.1 调参顺序

1. **验证舵机方向** - 推车倾斜时舵机应转向倾斜方向
2. **低速测试** - 从 2m/s 开始验证
3. **调整阻尼** - 修改 Q_phi_dot
4. **调整回正** - 修改 Q_delta
5. **扩展速度** - 逐步提高速度

---

6. 坐标系约定

```
        前进方向
           ↑
    左 ←──┼──→ 右
           |
        后

侧倾角 φ: 右倾为正
转向角 δ: 右转为正
```

Fence 8

平衡逻辑: 右倾 → 右转 → 离心力左推 → 回正

---

7. IMU/陀螺仪方向配置

7.1 需要哪个轴的数据?

LQR控制需要**车身侧倾角**（左右倾斜），即绕车身前进方向轴的旋转角度。

```
        前进方向 (X)
           ↑
           |     侧倾轴
    左 ←──●──→ 右
          /|\
         / | \  ← 这个旋转就是"侧倾"
        /  |  \
```

Fence 9

这个角度在IMU中可能叫 `roll` **或** `pitch`，取决于IMU的安装方向:

IMU安装方式	侧倾角对应	侧倾角速度对应
X轴朝前	`roll`	`gyro_x` 或 `roll_rate`
Y轴朝前	`pitch`	`gyro_y` 或 `pitch_rate`
X轴朝后	`-roll`	`-gyro_x`
Y轴朝后	`-pitch`	`-gyro_y`

Table 4

简单判断方法：把车向右倾斜，看哪个角度值变化明显，那个就是侧倾角。

7.2 坐标系说明

LQR模型内部（固定不变）：

- 侧倾角 φ：**右倾为正**
- 转向角 δ：**右转为正**

当前代码期望的IMU输入（默认配置）：

数据	正方向	负方向
`imu_get_roll()`	车身向**左**倾斜	车身向**右**倾斜
`imu_get_roll_rate()`	向**左**倾斜的角速度	向**右**倾斜的角速度

Table 5

> 函数名说明：代码中用 `imu_get_roll()` 只是一个命名，实际返回的应该是车身侧倾角，可能来自你IMU的roll或pitch，取决于安装方式。

为什么代码有负号？

```
phi = -(roll_deg - lqr_expect_phi) * DEG_TO_RAD;  // 负号做坐标系转换
phi_dot = -roll_rate_dps * DEG_TO_RAD;
```

Fence 10

代码假设IMU是「左倾为正」，通过负号转换成LQR需要的「右倾为正」。

7.3 如何判断方向是否正确

方法1：静态测试（推荐）

1. 将车辆扶正，记录 `imu_get_roll()` 返回值（应接近0）
2. 手动将车身向**右**倾斜
3. 观察 `imu_get_roll()` 返回值：
   - 数值**减小**（变负）→ 方向正确 ✓
   - 数值**增大**（变正）→ 方向反了 ✗

方法2：动态测试（最直观）

1. 让车静止或低速运行

2. 用手轻推车身向右倾斜

3. 观察舵机转向：

- 舵机向**右**转 → 整体方向正确 ✓
- 舵机向**左**转 → 方向反了 ✗

方法3：串口调试

```
// 在主循环中打印
printf("roll=%.2f, rate=%.2f\r\n", imu_get_roll(), imu_get_roll_rate());
```

Fence 11

向右倾斜时，roll 应该变负；向左倾斜时，roll 应该变正。

7.4 方向反了如何修改

在 `lqr_balance.c` 的 `lqr_balance_control()` 函数（第144-145行）中修改符号：

```
/* 坐标系对齐（根据实际情况调整符号） */
phi = -(roll_deg - lqr_expect_phi) * DEG_TO_RAD;      // 默认有负号
phi_dot = -roll_rate_dps * DEG_TO_RAD;                // 默认有负号
```

Fence 12

修改规则：

你的IMU方向	roll符号	roll_rate符号	修改方法
左倾为正（默认）	正确	正确	不修改
右倾为正	反了	反了	去掉两个负号
roll左正，rate右正	正确	反了	去掉 roll_rate 的负号
roll右正，rate左正	反了	正确	去掉 roll 的负号

Table 6

修改示例：如果你的IMU是「右倾为正」，改成：

```
phi = (roll_deg - lqr_expect_phi) * DEG_TO_RAD;       // 去掉负号
phi_dot = roll_rate_dps * DEG_TO_RAD;                 // 去掉负号
```

Fence 13

7.5 常见IMU安装情况

IMU芯片安装方向	通常roll方向	代码是否需要修改
芯片朝上，X轴朝前	左倾为正	不需要（默认）
芯片朝上，X轴朝后	右倾为正	去掉负号
芯片朝下	取决于X轴	需要测试确认

Table 7

7.6 快速验证清单

- ☐ 车身**右倾**时，`imu_get_roll()` 返回**负值**（如果是正值需要改代码）
- ☐ 车身**右倾**时，舵机向**右**打
- ☐ 车身从右倾**回正**时，`imu_get_roll_rate()` 返回**正值**
- ☐ 手推车身倾斜，舵机立即响应并转向倾斜方向

8. API 参考

8.1 应用层 (lqr_balance.h)

Table 8

函数	说明
`lqr_balance_init()`	初始化LQR平衡控制
`lqr_balance_control()`	主控制函数，5ms中断调用
`lqr_set_expect_phi(phi_deg)`	设置期望侧倾角（度），用于转弯
`lqr_get_velocity()`	获取当前速度估计
`lqr_tune_params(h, L)`	运行时调整物理参数

8.2 驱动层 (lqr_driver.h)

Table 9

函数	说明
`lqr_init(ctrl)`	初始化控制器结构体
`lqr_set_physical_params(ctrl, h, L, b)`	设置物理参数
`lqr_update_gain(ctrl, velocity)`	根据速度更新增益
`lqr_compute(ctrl, phi, phi_dot)`	执行LQR计算
`lqr_enable(ctrl, enable)`	使能/禁用控制器
`lqr_reset(ctrl)`	重置控制器状态

9. 需要用户实现的接口

以下函数在 `lqr_balance.h` 中声明为 `extern`，用户必须在自己的代码中实现：

```
// 必须实现
extern int motor_get_speed(void);        // 返回电机RPM
extern float imu_get_roll(void);         // 返回侧倾角（度）
extern float imu_get_roll_rate(void);    // 返回侧倾角速度（度/秒）
extern void servo_set(uint32_t pwm);     // 设置舵机PWM
```

10. 移植检查清单

- [ ] 复制 `driver/` 和 `app/` 到项目
- [ ] 实现 `motor_get_speed()`
- [ ] 实现 `imu_get_roll()` 和 `imu_get_roll_rate()`
- [ ] 实现 `servo_set()`
- [ ] 修改 `lqr_balance_init()` 中的物理参数
- [ ] 修改 `lqr_balance.h` 中的舵机PWM参数
- [ ] 验证 IMU 坐标系方向（可能需要调整符号）
- [ ] 验证舵机方向
- [ ] 设置定时中断调用 `lqr_balance_control()`

11. 依赖

- **驱动层**: 无依赖，纯 C 实现，仅需 `<stdint.h>`
- **应用层**: `<math.h>` (tanf)
- **工具**: Python 3 + numpy + scipy（tkinter 为 Python 标准库）

作者: 无心之举 | 2026